



Sort

Problem set



چند الگوریتمی که برای مرتب‌سازی اعداد در جلسه این هفته استفاده کردیم را به یاد بیاورید: مرتب‌سازی انتخابی (Selection sort)، مرتب‌سازی حبابی (Bubble sort) و مرتب‌سازی ادغامی (Merge sort).

- مرتب‌سازی انتخابی (Selection Sort) به شکل تکراری از بخش‌های مرتب‌نشده یک لیست عبور می‌کند، کوچک‌ترین عنصر را در هر مرحله انتخاب کرده و آن را به مکان درست خود منتقل می‌کند.
- مرتب‌سازی حبابی (Bubble Sort) مقادیر مجاور را به صورت جفت‌جفت مقایسه می‌کند و در صورتی که ترتیب آنها نادرست باشد، آنها را جابه‌جا می‌کند. این روند ادامه می‌یابد تا زمانی که لیست مرتب شود.
- مرتب‌سازی ادغامی (Merge Sort) به صورت بازگشتی لیست را به دو بخش تقسیم می‌کند و این کار را به طور مکرر انجام می‌دهد. سپس، لیست‌های کوچکتر را به ترتیب درست با هم ادغام کرده و لیست بزرگتری ایجاد می‌کند.

در این مسئله، شما سه برنامه مرتب‌سازی را تحلیل کرده و روش مرتب‌سازی‌ای که در آن‌ها استفاده را تشخیص می‌دهید.

در فایل به اسم `answers.txt` در فولدری به اسم `sort`، جواب‌های خود را به همراه توضیحات برای هر برنامه، با کامل کردن قسمت‌های `TODO`، بنویسید.

در این برنامه شما به مقداری `distribution code` نیاز دارید. این کدها توسط تیم CS50 نوشته شده‌اند. در اینجا سه کد کامپایل شده C وجود دارد: `sort1`، `sort2`، `sort3`، به همراه چندین فایل `.txt` برای ورودی و یک فایل `answers.txt`، برای نوشتن جواب‌هایتان. هر کدام از فایل‌های `sort1`، `sort2` و `sort3` با الگوریتم‌های مرتب‌سازی متفاوتی پیاده‌سازی شده‌اند: مرتب‌سازی انتخابی، مرتب‌سازی حبابی و مرتب‌سازی ادغامی (نه الزاماً با این ترتیب!). وظیفه شما، تشخیص الگوریتم استفاده شده در هر فایل می‌باشد. با دانلود کردن این فایل‌ها شروع کنید.

[VS Code](#) را باز کنید.

در ترمینال کلیک کنید و دستور `cd` را به تنهایی اجرا کنید. نتیجه باید به شکل زیر باشد:

```
$
```

در ترمینال کلیک کرده و دستور زیر را همراه با `Enter` اجرا کنید تا فایل `zip` به اسم `sort.zip` دانلود کنید. به فاصله بین `wget` و لینک دقت کنید و آن را عیناً کپی/پیست کنید.

```
wget https://cdn.cs50.net/2024/fall/psets/3/sort.zip
```

سپس دستور زیر را اجرا کنید تا فولدری به اسم `sort` ساخته شود:

```
unzip sort.zip
```

شما از اینجا به بعد دیگر به فایل `zip` نیاز ندارید پس با دستور زیر همراه با جواب `Y` می‌توانید آن را حذف کنید.

```
rm sort.zip
```

فایل‌های `.txt` را بررسی کنید.

- چندین فایل `.txt` در اختیار شما قرار داده شده است. این فایل‌ها حاوی n خط از مقادیر برعکس شده، به هم ریخته یا مرتب‌شده است.
برای مثال، فایل `reversed10000.txt` دارای 10000 خط از اعداد برعکس شده از 1 تا 10000 است، در حالی که `random50000.txt`، دارای 50000 خط از اعداد که به صورت اتفاقی چیده شده‌اند است.
- انواع مختلف فایل‌های `.txt` ممکن است به شما کمک کنند که مشخص کنید کدام مرتب‌سازی مربوط به کدام نوع است. در نظر داشته باشید که هر کدام از الگوریتم‌ها روی یک لیست مرتب شده چگونه عمل می‌کنند. روی یک لیست برعکس شده چگونه؟ یک لیست به هم ریخته چگونه؟ ممکن است مفید باشد که یک لیست کوچک از هر نوع را بررسی کرده و فرآیند مرتب‌سازی هر کدام را قدم به قدم پیش ببرید.

برای هر مرتب‌سازی با ورودی‌های مختلف زمان‌سنجی کنید.

- برای استفاده از مرتب‌سازی‌های داخل فایل‌های `.txt` ، دستور `./[program_name] [text_file.txt]` را در ترمینال اجرا کنید. مطمئن شوید که با `cd` به فولدر `sort` انتقال یافته‌اید!

برای مثال برای مرتب‌سازی `reversed10000.txt` با `sort1`، باید دستور `./sort1 reversed10000.txt` را اجرا کنید.

- شاید زمان‌سنجی مرتب‌سازی‌ها هم مفید باشد، برای این کار دستور `time ./[sort_file] [text_file.txt]` را اجرا کنید.

برای مثال می‌توانید با اجرا کردن `time ./sort1 reversed10000.txt` ، `sort1` را روی 10000 عدد برعکس شده اجرا کنید. در انتهای خروجی ترمینال خود، می‌توانید به زمان `real` نگاه کنید تا ببینید در حین اجرای برنامه چه مدت زمانی سپری شده است.

درستی

```
check50 cs50/problems/2025/x/sort
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/sort
```


CS50x Iran

Harvard's Computer Science 50x Iran

